

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Beffy A Shanika	Reg. No.:	192511387
Section / Batch:	2025	Date:	27/07/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Q1. A delivery system calculates distances between drop points to identify the nearest locations using brute-force. a) Explain how all pairs are evaluated. b) Derive the time complexity. c) Discuss scalability concerns.

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: beffy

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Delivery System Using Brute-Force to Find Nearest Drop Points

CODE:

```
import math

# Function to calculate Euclidean distance

def distance(p1, p2):

    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

# Coordinates of drop points

points = [(2, 3), (5, 7), (8, 2), (6, 5)]

min_dist = float('inf')

nearest_pair = ()

# Brute-force: Check every pair

for i in range(len(points)):

    for j in range(i + 1, len(points)):

        d = distance(points[i], points[j])

        print(f"Distance between {points[i]} and {points[j]} = {d:.2f}")

        if d < min_dist:

            min_dist = d

            nearest_pair = (points[i], points[j])

print("\nNearest Drop Points:", nearest_pair)

print("Minimum Distance:", round(min_dist, 2))
```

OUTPUT:

```
Distance between (2, 3) and (5, 7) = 5.00
Distance between (2, 3) and (8, 2) = 6.08
Distance between (2, 3) and (6, 5) = 4.47
Distance between (5, 7) and (8, 2) = 5.83
Distance between (5, 7) and (6, 5) = 2.24
Distance between (8, 2) and (6, 5) = 3.61

Nearest Drop Points: ((5, 7), (6, 5))
Minimum Distance: 2.24
```